

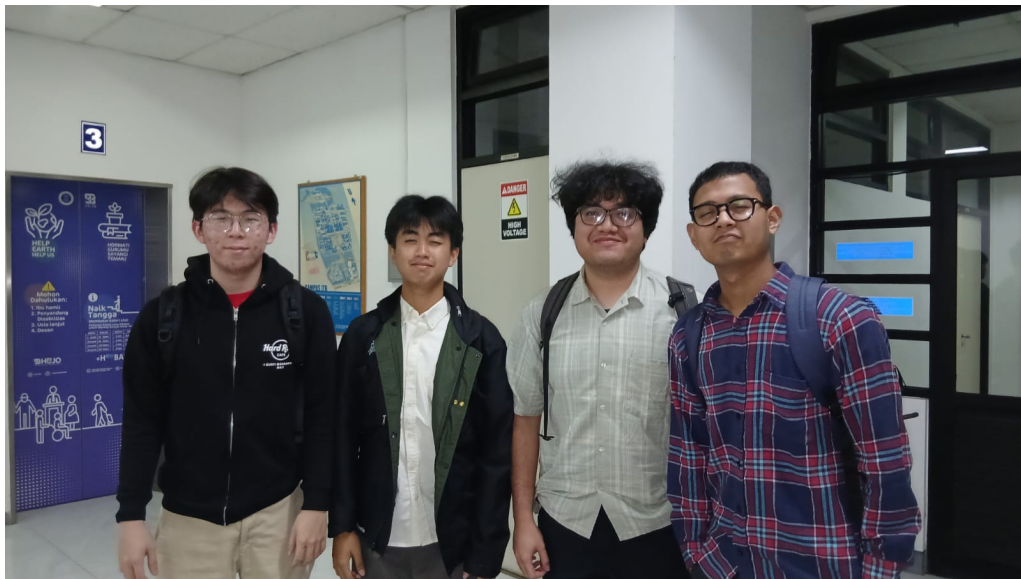
Tugas Besar

Milestone 3

IF2224 Teori Bahasa Formal dan Otomata

Semantic Analysis

Semester 2 Tahun 2025/2026



Disusun oleh:

Kai

Jonathan Kris Wicaksono	(13524023)
Vincent Rionarlie	(13524031)
Muhammad Aufar Rizqi Kusuma	(13524061)
Bryan Pratama Putra Hendra	(13524067)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	4
1.1	Latar Belakang	4
1.2	Rumusan Masalah	4
1.3	Tujuan	5
1.4	Batasan Masalah	5
2	Dasar Teori	6
2.1	Semantic Analysis	6
2.2	Abstract Syntax Tree	6
2.3	Decorated Abstract Syntax Tree	8
2.4	Symbol Table	9
2.4.1	Tabel tab	9
2.4.2	Tabel bt ab	10
2.4.3	Tabel a tab	10
2.4.4	Manajemen Scope dan Lookup	10
2.5	Type Compatibility	12
2.5.1	Simple Type	12
2.5.2	Structured Type	12
2.5.3	Assignment Compatibility	13
2.5.4	Ringkasan Operator dan Hasil Tipe	13
2.6	Konversi Parse Tree ke AST	15
2.6.1	Syntax-Directed Translation	15
2.6.2	Semantic Action	16
2.7	Predefined Identifier	16
2.8	Semantic Error Handling	17
3	Perancangan dan Implementasi	18
3.1	Gambaran Umum Program	18
3.2	Struktur Direktori Program	18
3.3	Perancangan AST	19
3.4	Syntax-Directed Translation	19
3.5	Implementasi Symbol Table	20
3.6	Implementasi Semantic Analyzer	20
3.6.1	Scope dan Block	21
3.6.2	Type Resolution	21
3.6.3	Type Checking	21



3.6.4	Procedure dan Function	21
3.7	Format Output	21
4	Pengujian	22
4.1	Strategi Pengujian	22
4.2	Detail Pengujian	23
4.2.1	Kasus 1: Program Dasar	23
4.2.2	Kasus 2: Ekspresi dan Boolean	24
4.2.3	Kasus 3: Control Flow	25
4.2.4	Kasus 4: Structured Type	26
4.2.5	Kasus 5: Subprogram	27
4.3	Pengujian Error Semantic	29
5	Penutup	30
5.1	Kesimpulan	30
5.2	Saran	30
6	Lampiran	31
6.1	Tautan GitHub	31
6.2	Tabel Kontribusi	32
6.3	Cara Menjalankan Program	32



Daftar Algoritma

1	Manajemen scope dan lookup	11
2	Type checking ekspresi biner	14
3	Konversi ringkas parse tree ke AST	15

Bab 1

Pendahuluan

1.1 Latar Belakang

Compiler merupakan perangkat lunak yang menerjemahkan program dari bahasa tingkat tinggi menjadi representasi yang dapat diproses lebih lanjut oleh mesin. Tahap awal compiler tidak langsung menghasilkan kode mesin, melainkan membentuk representasi internal yang semakin kaya. Lexical analysis mengubah karakter menjadi token, syntax analysis memeriksa struktur token berdasarkan grammar, dan semantic analysis memeriksa makna program yang tidak dapat ditentukan hanya dari grammar.

Pada Milestone 1, Arion Compiler telah memiliki lexer yang mampu menghasilkan token stream. Pada Milestone 2, token tersebut diproses oleh Recursive Descent Parser untuk menghasilkan parse tree. Parse tree menunjukkan bahwa program valid secara sintaksis, tetapi belum menjamin bahwa program bermakna benar. Program seperti assignment string ke integer, pemakaian identifier yang belum dideklarasikan, atau pemanggilan fungsi dengan parameter yang salah dapat saja memiliki bentuk sintaks yang valid, tetapi tetap salah secara semantik.

Milestone 3 berfokus pada semantic analysis. Tahap ini mengubah parse tree menjadi Abstract Syntax Tree (AST), mengelola symbol table, melakukan type checking, mengatur scope, dan memberi anotasi pada AST sehingga setiap node memiliki informasi semantik seperti tipe data, lexical level, block index, dan referensi ke symbol table. Hasil akhirnya adalah decorated AST beserta tiga tabel simbol, yaitu `tab`, `btabs`, dan `atab`.

1.2 Rumusan Masalah

Permasalahan utama pada Milestone 3 adalah bagaimana membangun semantic analyzer yang dapat memanfaatkan parse tree hasil Milestone 2 untuk menghasilkan representasi yang lebih ringkas dan bermakna. Permasalahan tersebut diuraikan menjadi beberapa pertanyaan berikut.

1. Bagaimana mengonversi parse tree menjadi AST menggunakan prinsip Syntax-Directed Translation?

2. Bagaimana merancang struktur AST yang cukup ekspresif untuk merepresentasikan deklarasi, statement, ekspresi, procedure, function, array, record, dan control flow?
3. Bagaimana mengelola symbol table `tab`, `btab`, dan `atab` untuk menyimpan identifier, block, scope, dan informasi array?
4. Bagaimana melakukan lookup identifier, pengecekan redeklarasasi, type checking, assignment compatibility, dan validasi control flow?
5. Bagaimana menampilkan output semantic analysis dalam bentuk symbol table dan decorated AST yang mudah diperiksa?

1.3 Tujuan

Tujuan pengerjaan Milestone 3 adalah membangun tahap semantic analysis pada Arion Compiler. Secara khusus, tujuan yang ingin dicapai adalah sebagai berikut.

1. Membuat AST dari parse tree hasil parser.
2. Menghilangkan detail sintaksis yang tidak diperlukan pada tahap semantik, seperti keyword struktural dan tanda baca.
3. Mengimplementasikan symbol table `tab`, `btab`, dan `atab`.
4. Menginisialisasi predefined identifier seperti `Integer`, `Real`, `Char`, `Boolean`, `String`, `True`, `False`, `writeln`, dan `readln`.
5. Melakukan pengecekan deklarasi, scope, tipe ekspresi, assignment, parameter procedure/function, array access, record field access, serta kondisi pada statement kontrol.
6. Menghasilkan decorated AST dan symbol table sebagai output program.

1.4 Batasan Masalah

Implementasi Milestone 3 dibatasi pada frontend compiler Arion. Program tidak melakukan code generation atau eksekusi program Arion. Semantic analyzer bekerja pada AST yang dibangun dari parse tree hasil parser, sehingga validitas sintaks diasumsikan sudah diperiksa oleh tahap sebelumnya.

Pengecekan semantik yang diimplementasikan berfokus pada deklarasi identifier, scope, type checking, assignment compatibility, procedure/function call, array, record, subrange, enumerated type, dan validasi kondisi control flow. Fitur runtime seperti nilai aktual variabel saat program berjalan tidak dievaluasi secara penuh. Program juga belum melakukan analisis data-flow lengkap untuk mendeteksi seluruh kemungkinan penggunaan variabel sebelum inisialisasi.

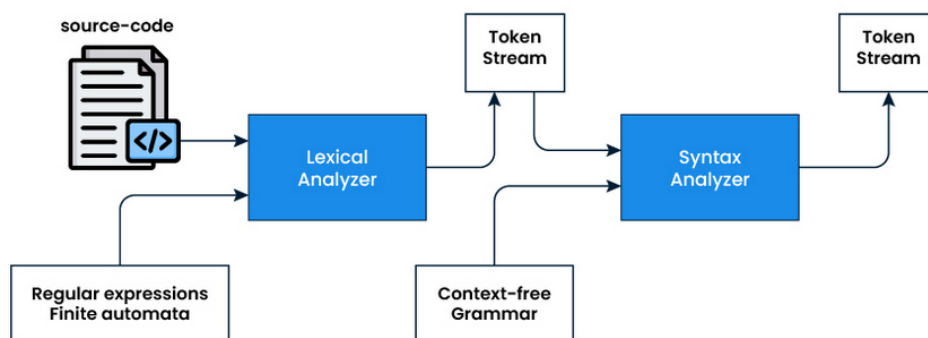
Bab 2

Dasar Teori

2.1 Semantic Analysis

Semantic analysis adalah tahap analisis pada compiler yang memeriksa kesesuaian makna program setelah strukturnya dinyatakan valid. Berbeda dari analisis sintaks yang berfokus pada bentuk, tahap ini menilai apakah konstruksi program memenuhi aturan semantik bahasa, misalnya kesesuaian tipe operand, keabsahan target assignment, dan keterdeklarasian identifier sebelum digunakan [1, 3, 4].

Dalam arsitektur compiler modern, semantic analysis umumnya dilakukan setelah parsing dan sebelum pembentukan representasi antara. Hasil tahap ini berupa informasi tambahan yang melekat pada node sintaks, seperti tipe, scope, dan referensi ke simbol yang terkait.



Gambar 2.1: Alur umum analisis semantik dalam compiler

Gambar 2.1 menunjukkan bahwa semantic analysis berada di antara parser dan tahap pembentukan representasi program yang lebih tinggi. Pada posisi ini, compiler mulai menggabungkan struktur sintaks dengan informasi semantik.

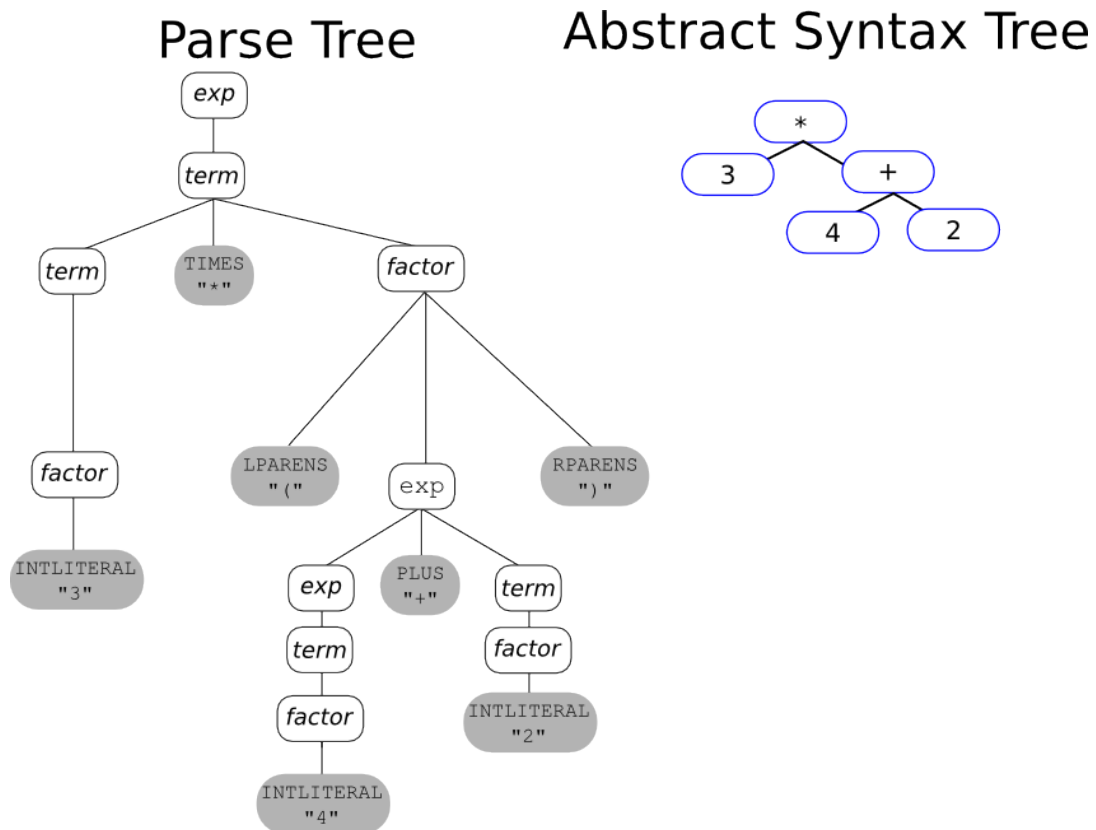
2.2 Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah representasi pohon yang menyederhanakan struktur program dengan mempertahankan hanya unsur yang relevan secara se-

mantik. Jika parse tree memuat detail grammar secara lengkap, AST biasanya menghilangkan perantara dan simbol sintaksis yang tidak diperlukan untuk analisis lanjutan [5, 2].

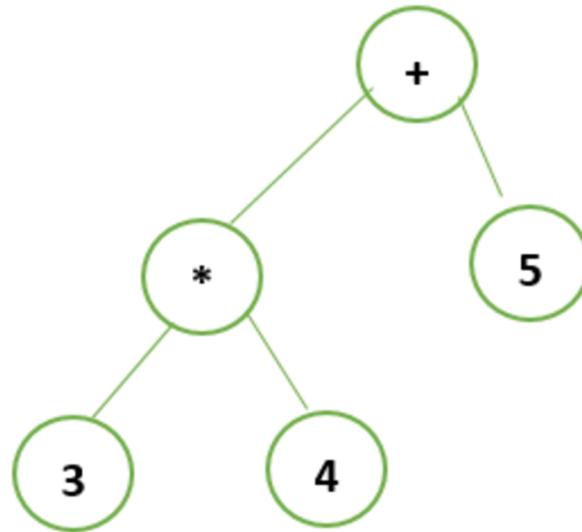
Secara umum, AST memudahkan proses analisis berikutnya karena relasi antarooperasi dan operand menjadi lebih jelas. Representasi ini juga lebih ringkas sehingga lebih efisien untuk ditelusuri dibanding parse tree yang lengkap.

Sebagai ilustrasi, ekspresi penugasan dapat direpresentasikan sebagai node penugasan yang memiliki anak berupa target dan nilai, sedangkan ekspresi aritmetika direpresentasikan sebagai node operator biner dengan dua operand.



Gambar 2.2: Perbandingan parse tree dan abstract syntax tree

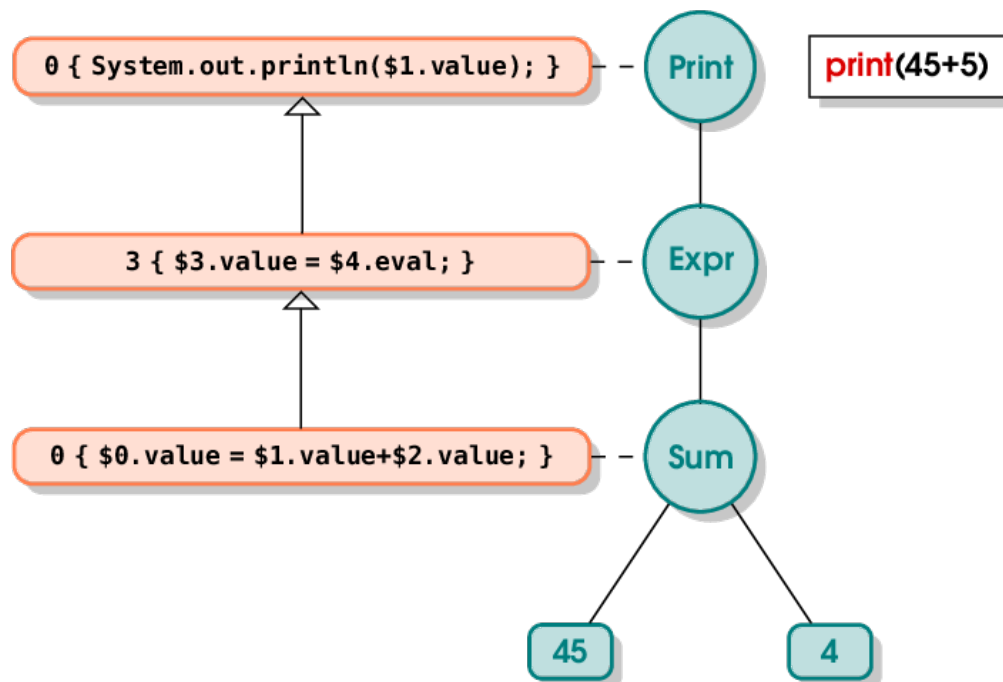
Ilustrasi pada Gambar 2.3 menunjukkan bahwa satu pernyataan sederhana pun dapat menghasilkan banyak node perantara dalam parse tree. Kondisi ini menjelaskan mengapa AST biasanya dipilih sebagai representasi yang lebih efisien untuk analisis lanjutan.



Gambar 2.3: Contoh parse tree untuk deklarasi sederhana

2.3 Decorated Abstract Syntax Tree

Decorated AST adalah AST yang telah diperkaya dengan informasi hasil analisis semantik. Pada tahap ini, node AST tidak hanya menyimpan struktur program, tetapi juga atribut tambahan yang dibutuhkan untuk pemeriksaan lebih lanjut.



Gambar 2.4: Contoh decorated abstract syntax tree

- tipe hasil inferensi atau verifikasi pada node
- referensi ke simbol yang terkait
- informasi scope atau lexical level
- atribut tambahan yang diperlukan oleh tahap lanjutan.

Decorated AST menunjukkan struktur program sekaligus hasil validasi semantiknya. Karena itu, representasi ini sering digunakan sebagai landasan untuk pembentukan intermediate representation atau tahap generasi kode [3, 2].

2.4 Symbol Table

Symbol table adalah struktur data yang menyimpan informasi identifier beserta konteks kemunculannya dalam program. Dalam compiler, struktur ini digunakan untuk mendukung deklarasi, lookup, tipe data, dan pengelolaan scope.

2.4.1 Tabel tab

Salah satu tabel utama menyimpan entri identifier seperti nama program, konstanta, variabel, tipe, prosedur, dan fungsi. Setiap entri umumnya memuat nama simbol, kategori objek, tipe, level scope, serta relasi ke entri lain dalam scope yang sama.

2.4.2 Tabel btab

Tabel lain dapat digunakan untuk menyimpan informasi block atau scope, seperti batas awal dan akhir scope, parameter, serta ukuran variabel lokal. Informasi ini membantu compiler menjaga keterkaitan antara deklarasi dan ruang lingkup kemunculan identifier.

2.4.3 Tabel atab

Jika bahasa mendukung tipe terstruktur seperti array, compiler lazim menyimpan atribut tambahan pada tabel terpisah, misalnya tipe indeks, tipe elemen, dan batas rentang. Pemisahan atribut semacam ini memudahkan pemeriksaan kesesuaian tipe dan perhitungan ukuran data.

2.4.4 Manajemen Scope dan Lookup

Manajemen scope dan mekanisme lookup adalah bagian krusial dari semantic analysis karena menentukan relasi antara nama (identifier) dan entri deklarasinya. Secara formal, compiler memelihara konsep lingkungan (environment) yang merepresentasikan kumpulan frame scope yang aktif pada titik tertentu dalam program. Kita dapat menulis environment sebagai urutan frame

$$E = [F_n, F_{n-1}, \dots, F_0]$$

di mana setiap frame F_i adalah peta (mapping) dari identifier ke informasi deklarasi (misalnya tipe, lokasi, atribut parameter). Frame teratas F_n merepresentasikan scope terdalam saat ini, sedangkan F_0 biasanya adalah scope global.

Operasi dasar pada manajemen scope adalah sebagai berikut.

1. Masuk scope (enter scope): menambahkan frame kosong di puncak environment sehingga deklarasi selanjutnya ditempatkan pada frame baru tersebut.
2. Keluar scope (exit scope): menghapus frame teratas dari environment sehingga deklarasi lokal tidak lagi terlihat.
3. Deklarasi (declare): menambahkan pasangan (name $\mapsto$ info) ke frame teratas; jika nama sudah ada pada frame yang sama, biasanya dianggap error redeclaration.
4. Pencarian (lookup): mencari nama dimulai dari frame teratas ke frame yang lebih bawah hingga ditemukan atau sampai seluruh environment habis.

Dalam notasi fungsi, lookup dapat dipandang sebagai

$$\text{lookup}(E, x) = \begin{cases} F_k(x) & \text{jika } k = \min\{i \mid F_i(x) \text{ terdefinisi}\}, \\ \perp & \text{jika tidak ditemukan} \end{cases}$$

di mana $F_i(x)$ adalah hasil pencarian identifier x pada frame F_i , dan $\perp$ menandakan kegagalan lookup (undeclared identifier). Relasi ini menegaskan sifat terarah dari pencarian: selalu mulai dari scope terdalam. Beberapa poin penting yang perlu diperhatikan

1. Shadowing: ketika sebuah deklarasi baru dengan nama yang sama muncul di scope terdalam, deklarasi tersebut menutupi (shadow) deklarasi dengan nama sama pada scope lebih luar. Shadowing adalah fenomena yang sah di banyak bahasa dan harus ditangani oleh rule lookup.
2. Redclaration error: redeklarasi pada scope yang sama umumnya ditolak; ini dicegah dengan mengecek keberadaan nama pada frame teratas sebelum menambahkan entri baru.
3. Lexical (static) vs dynamic scoping: kebanyakan bahasa modern memakai lexical scoping, di mana lingkungan lookup ditentukan oleh struktur statis program, bukan urutan pemanggilan runtime. Implementasi yang membedakan kedua model hendaknya menegaskan asumsi scoping yang dipakai oleh compiler.
4. Lexical (static) vs dynamic scoping: kebanyakan bahasa modern memakai lexical scoping, di mana lingkungan lookup ditentukan oleh struktur statis program, bukan urutan pemanggilan runtime. Implementasi yang membedakan kedua model hendaknya menegaskan asumsi scoping yang dipakai oleh compiler.

Penjabaran formal dan prosedural ini menjadi dasar bagi implementasi praktis seperti tabel-tabel `tab`, `btab`, dan `stack scope` yang dipakai pada banyak compiler klasik. Berikut pseudocode ringkas yang menggambarkan operasi-operasi tersebut.

Algoritma 1: Manajemen scope dan lookup

```
1: procedure ENTERSCOPE
2:   btab.append({last = currentLast, lpar = 0, psze = 0, vsze = 0})
3:   push(scopeLastStack, 0)
4: end procedure
5: procedure EXITSCOPE
6:   pop(scopeLastStack)
7: end procedure
8: procedure DECLARE(id, obj, typ, adr)
9:   if lookupCurrentScope(id) exists then
10:    return error("redeclared identifier")
11:   end if
12:   entry = {identifier = id, link = currentLast, obj = obj, type =
      typ,
      nrm = 1, lev = level, adr = adr}
13:   index = tab.append(entry)
14:   currentLast = index
15: end procedure
16: procedure LOOKUP(id)
17:   for scopeLast in scopeLastStack from top do
18:     idx = scopeLast
19:     while idx  $\neq$  0 do
```

```
20:         if tab[idx].identifier == id then
21:             return tab[idx]
22:         end if
23:         idx = tab[idx].link
24:     end while
25: end for
26: return error("undeclared identifier")
27: end procedure
```

2.5 Type Compatibility

Type compatibility menentukan apakah dua tipe dapat digunakan bersama dalam ekspresi, penugasan, atau pemanggilan prosedur. Konsep ini penting karena compiler perlu membedakan operasi yang sah secara sintaks tetapi tidak sah secara semantik [3, 6].

2.5.1 Simple Type

Secara umum, tipe sederhana mencakup tipe numerik, karakter, Boolean, string, serta tipe turunan seperti subrange dan enumerated type:

1. **Real**: `realcon` memiliki type `Real`.
2. **Integer**: `intcon` memiliki type `Integer`.
3. **Char**: `charcon` memiliki type `Char`.
4. **Boolean**: ekspresi yang dibentuk oleh `notsy`, `andsy`, `orsy`, dan relational operator menghasilkan Boolean.
5. **String**: `string` memiliki type `String`.
6. **Subrange**: type ditentukan oleh constant di dalam range, dan batas bawah tidak boleh lebih besar dari batas atas.
7. **Enumerated**: type ditentukan oleh ident di dalam enumerated dan seluruh ident di dalamnya harus bertipe sama.

2.5.2 Structured Type

Structured type dapat bersifat *named* jika didefinisikan melalui deklarasi tipe, atau *anonymous* jika dibentuk langsung pada deklarasi variabel.

- **Array**: memiliki elemen yang diakses melalui indeks dengan domain yang terdefinisi.
- **Record**: mengelompokkan beberapa field dengan tipe yang dapat berbeda.

Dalam pendekatan nominal typing, dua tipe terstruktur yang tampak sama belum tentu kompatibel bila identitas tipenya berbeda.

2.5.3 Assignment Compatibility

Assignment compatibility menjelaskan apakah suatu nilai bertipe tertentu dapat ditempatkan ke dalam lokasi penyimpanan dengan tipe target yang lain. Relasi ini tidak selalu sama dengan kesamaan tipe, karena banyak bahasa pemrograman memperbolehkan konversi implisit pada kondisi tertentu. Pemeriksaan assignment tidak hanya membandingkan label tipe, tetapi juga mempertimbangkan apakah bahasa mengizinkan *widening conversion*, *subset relation*, atau aturan khusus untuk tipe terstruktur dan string. Secara umum, hubungan ini dapat dinyatakan sebagai berikut.

$$\text{assignable}(T_2, T_1)$$

Notasi tersebut dibaca sebagai "nilai bertipe T_2 dapat ditugaskan ke target bertipe T_1 ". Relasi ini bersifat terarah. Artinya, fakta bahwa T_2 dapat di-assign ke T_1 tidak otomatis berarti sebaliknya juga berlaku. Dalam praktiknya, compiler perlu memeriksa baik tipe sumber maupun tipe target, lalu menentukan apakah penugasan itu aman atau memerlukan konversi eksplisit. Beberapa aturan yang umum dipakai dapat dirangkum sebagai berikut.

- Jika T_1 dan T_2 identik, assignment umumnya valid tanpa konversi tambahan.
- Jika bahasa mendukung konversi numerik, nilai bertipe integer sering dapat ditugaskan ke real.
- Jika nilai sumber merupakan anggota himpunan nilai yang lebih sempit, assignment hanya valid bila target memang menerima himpunan tersebut.
- Untuk tipe terstruktur, kompatibilitas biasanya bergantung pada identitas tipe atau aturan kesetaraan struktur yang didefinisikan oleh bahasa.

Dalam analisis semantik, assignment compatibility dipakai pada beberapa konteks sekaligus, bukan hanya assignment statement. Pemeriksaan ini juga muncul pada argumen aktual saat pemanggilan prosedur atau fungsi, pada inisialisasi variabel, serta pada ekspresi yang melibatkan operator tertentu. Karena itu, pembahasan assignment compatibility menjadi salah satu komponen penting dalam type checking: kesalahan pada tahap ini sering kali lolos dari analisis sintaks, tetapi dapat menyebabkan perilaku program yang tidak sesuai jika tidak ditangkap oleh compiler.

Pada implementasi compiler yang mendukung tipe turunan, aturan assignment compatibility biasanya dipandang sebagai kombinasi tiga lapis pemeriksaan, yaitu kesamaan atau kompatibilitas dasar tipe, izin konversi implisit, dan pembatasan khusus dari bahasa terhadap tipe tertentu seperti array, record, subrange, enumerated type, atau string. Dengan pendekatan ini, compiler dapat membedakan penugasan yang valid secara semantik dari penugasan yang hanya tampak benar secara sintaks.

2.5.4 Ringkasan Operator dan Hasil Tipe

Aturan type checking sering diringkas dalam bentuk tabel operator agar hasil inferensi tipe konsisten saat anotasi AST.

Tabel 2.1: Ringkasan operator dan hasil tipe

Operator	Operand yang valid	Hasil
$+, -, *$	Integer/Real (kombinasi numerik)	Integer atau Real
$/$	Integer/Real (kombinasi numerik)	Real
div, mod	Integer $\times$ Integer	Integer
$\text{and}, \text{or}, \text{not}$	Boolean	Boolean
$=, <>, <, <=, >, >=$	Tipe kompatibel	Boolean

Algoritma berikut merangkum prosedur yang dipakai untuk menentukan tipe hasil operasi biner pada AST. Algoritma ini menerima operator dan tipe kedua operand, menerapkan aturan numerik, logika, dan relasional yang ditetapkan bahasa, serta mengembalikan tipe hasil atau error ketika operand tidak sesuai. Hasil penentuan tipe ini digunakan oleh semantic analyzer untuk menandai node AST dan menangkap kesalahan tipe pada waktu kompilasi.

Algoritma 2: Type checking ekspresi biner

```
1: procedure TYPEOFBINARYOP(op, left, right)
2:    $t_l \leftarrow \text{TYPEOF}(\textit{left})$ 
3:    $t_r \leftarrow \text{TYPEOF}(\textit{right})$ 
4:   if  $op \in \{+, -, *\}$  then
5:     if  $t_l = \textit{Integer}$  and  $t_r = \textit{Integer}$  then
6:       return Integer
7:     else if  $t_l \in \{\textit{Integer}, \textit{Real}\}$  and  $t_r \in \{\textit{Integer}, \textit{Real}\}$  then
8:       return Real
9:     end if
10:    return error("numeric operands required")
11:  end if
12:  if  $op = /$  then
13:    if  $t_l \in \{\textit{Integer}, \textit{Real}\}$  and  $t_r \in \{\textit{Integer}, \textit{Real}\}$  then
14:      return Real
15:    end if
16:    return error("numeric operands required")
17:  end if
18:  if  $op \in \{\textit{div}, \textit{mod}\}$  then
19:    if  $t_l = \textit{Integer}$  and  $t_r = \textit{Integer}$  then
20:      return Integer
21:    end if
22:    return error("integer operands required")
23:  end if
24:  if  $op \in \{\textit{and}, \textit{or}\}$  then
25:    if  $t_l = \textit{Boolean}$  and  $t_r = \textit{Boolean}$  then
26:      return Boolean
27:    end if
```

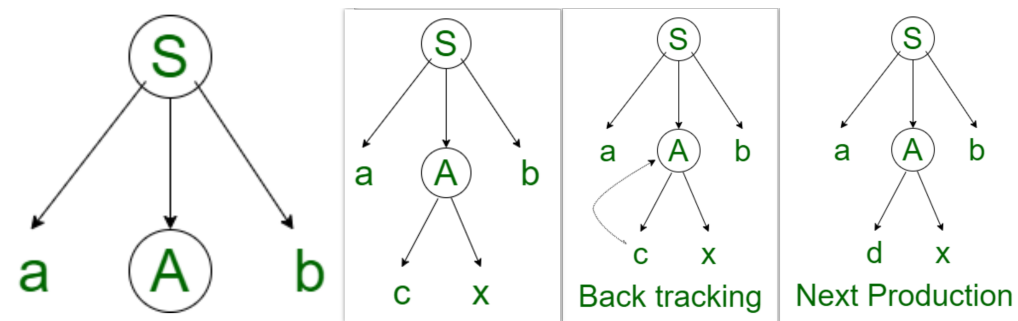
```

28:     return error("boolean operands required")
29: end if
30: if  $op \in \{<, \leq, >, \geq, =, \neq\}$  then
31:     if COMPATIBLE( $t_l, t_r$ ) then
32:         return Boolean
33:     end if
34:     return error("incompatible relational operands")
35: end if
36: end procedure

```

2.6 Konversi Parse Tree ke AST

Transformasi parse tree menjadi AST umumnya dilakukan melalui syntax-directed translation, yaitu pendekatan yang mengaitkan produksi grammar dengan atribut atau aksi semantik tertentu. Tujuannya adalah menghilangkan detail sintaksis yang tidak relevan dan mempertahankan struktur yang penting untuk analisis lanjutan [2, 5].



Gambar 2.5: Ilustrasi proses penurunan sintaks pada parser rekursif-descent

Gambar 2.5 memperlihatkan bahwa parser rekursif-descent membangun struktur sintaks melalui ekspansi produksi, termasuk kemungkinan backtracking pada kondisi tertentu. Mekanisme ini berkaitan langsung dengan pembentukan parse tree yang kemudian dapat disederhanakan menjadi AST.

2.6.1 Syntax-Directed Translation

Pada pendekatan bottom-up, subtree anak dibentuk terlebih dahulu, kemudian hasilnya digabung pada node induk. Dengan cara ini, semantic action dapat dipandang sebagai fungsi yang mengubah subtree menjadi representasi AST yang lebih ringkas.

Algoritma 3: Konversi ringkas parse tree ke AST

```

1: procedure ASSIGNMENT( $id, exprNode$ )
2:     return Assign(Var( $id$ ),  $exprNode$ )
3: end procedure
4: procedure BINARYEXPRESSION( $left, op, right$ )

```



```
5:   return BinOp(op, left, right)
6: end procedure
7: procedure FACTOR(value)
8:   if value is NUMBER then
9:     return Number(value)
10:  else if value is IDENT then
11:    return Var(value)
12:  end if
13: end procedure
```

2.6.2 Semantic Action

Semantic action adalah aturan tambahan yang dilekatkan pada produksi grammar untuk membentuk node AST atau menghitung atribut semantik. Contoh umum penggunaan semantic action ditunjukkan pada tabel berikut:

Tabel 2.2: Contoh semantic action pada grammar

Production Rule	Semantic Action
<code><assignment-statement> -> ID := <expr></code>	<code>AssignNode(VarNode(ID), expr.node)</code>
<code><expression> -> <simple-expr> op <term></code>	<code>BinOpNode(op, simple.node, term.node)</code>
<code><expression> -> <simple-expr></code>	<code>expression.node = simple.node</code>
<code><factor> -> NUMBER</code>	<code>NumberNode(NUMBER.value)</code>
<code><factor> -> ID</code>	<code>VarNode(ID.lexeme)</code>
<code><procedure-call> -> ID (<params>)</code>	<code>ProcCallNode(ID.lexeme, params.nodes)</code>

2.7 Predefined Identifier

Pada banyak compiler, symbol table diinisialisasi dengan identifier bawaan sebelum program pengguna dianalisis. Identifier ini biasanya mencakup tipe dasar, literal Boolean, serta prosedur atau fungsi I/O yang tersedia sejak awal [6, 1].

Keberadaan predefined identifier membantu semantic analyzer mengenali simbol bawaan tanpa menunggu deklarasi dari program sumber. Predefined identifier biasanya diinisialisasi segera setelah struktur symbol table dibuat. Tujuannya:

- menyediakan tipe-tipe dasar (mis. `Integer`, `Real`, `Char`, `Boolean`, `String`) sehingga node AST dapat diberi anotasi tipe;
- mendaftarkan konstanta bawaan (mis. `True`, `False`) untuk validasi literal Boolean;
- menambahkan prosedur/fungsi runtime yang tersedia (mis. `readln`, `writeln`) sehingga pemanggilan dapat divalidasi tanpa deklarasi eksplisit; dan

- menetapkan atribut khusus (mis. ukuran tipe, parameter fungsi, aritas) yang dipakai oleh semantic analyzer.

Contoh inisialisasi dari predefined identifier adalah sebagai berikut ini.

- `tab[1] = {identifier="Integer", kind=type, size=4}`
- `tab[2] = {identifier="Real", kind=type, size=8}`
- `tab[3] = {identifier="True", kind=const, type=Integer}`
- `tab[4] = {identifier="False", kind=const, type=Integer}`
- `tab[5] = {identifier="writeln", kind=proc, params=[vararg], return=void}`

2.8 Semantic Error Handling

Semantic error muncul ketika program lolos pemeriksaan sintaks tetapi melanggar aturan makna bahasa. Contoh yang umum adalah redeclaration pada scope yang sama, penggunaan identifier yang belum dideklarasikan, penugasan dengan tipe yang tidak kompatibel, dan penerapan operator pada operand yang tidak sesuai. Pesan error sebaiknya cukup informatif agar lokasi dan penyebab kesalahan mudah ditelusuri [4].

- Redeclaration error: identifier dideklarasikan dua kali pada scope yang sama.
- Undeclared identifier error: identifier dipakai tetapi tidak ditemukan pada scope mana pun.
- Type mismatch error: tipe ekspresi tidak cocok dengan tipe target assignment.
- Invalid operand error: operator dipakai pada tipe operand yang tidak sesuai.

Bab 3

Perancangan dan Implementasi

3.1 Gambaran Umum Program

Implementasi Milestone 3 melanjutkan pipeline Milestone 1 dan Milestone 2. Alur utama berada pada `src/main.cpp`. Program membaca file input, lalu menentukan bentuk input tersebut. Jika input adalah parse tree, program menggunakan `ParseTreeReader`. Jika input adalah token stream, program menggunakan `TokenStreamReader` lalu menjalankan parser. Jika input adalah source code Arion, program menjalankan lexer dan parser.

Setelah parse tree tersedia, program menjalankan `ASTBuilder` untuk membentuk AST. AST tersebut kemudian dianalisis oleh `SemanticAnalyzer`. Jika tidak ada error, program mencetak `tab`, `btabs`, `atab`, parse tree, dan decorated AST.

Pipeline Program

```
Input file
-> Lexer / TokenStreamReader / ParseTreeReader
-> Parser
-> Parse Tree
-> ASTBuilder
-> AST
-> SemanticAnalyzer
-> Symbol Table + Decorated AST
```

3.2 Struktur Direktori Program

Bagian Milestone 3 terutama berada pada direktori `src/semantic/`. Berkas utama yang digunakan adalah sebagai berikut.

- `ast_node.h` dan `ast_node.cpp`: definisi hierarchy node AST dan struktur anotasi.
- `ast_builder.h` dan `ast_builder.cpp`: konversi parse tree menjadi AST dengan Syntax-Directed Translation.

- `ast_formatter.h` dan `ast_formatter.cpp`: pencetakan decorated AST.
- `symbol_table.h` dan `symbol_table.cpp`: implementasi `tab`, `btabs`, `atab`, `scope stack`, registrasi identifikasi, dan `lookup`.
- `semantic_analyzer.h` dan `semantic_analyzer.cpp`: traversal AST, type checking, scope checking, dan anotasi node.
- `parse_tree_reader.h` dan `parse_tree_reader.cpp`: pembacaan parse tree hasil Milestone 2 sebagai input Milestone 3.

3.3 Perancangan AST

AST dirancang sebagai hierarchy class berbasis polymorphism. Semua node mewarisi `ASTNode`, yang memiliki field `annotation`. Node AST yang digunakan mencakup `ProgramNode`, `VarDeclNode`, `ConstDeclNode`, `TypeDeclNode`, `ProcedureDeclNode`, `FunctionDeclNode`, `AssignNode`, `BinOpNode`, `UnaryOpNode`, `VarRefNode`, `ArrayAccessNode`, `FieldAccessNode`, `ProcCallNode`, `FuncCallNode`, `IfNode`, `WhileNode`, `RepeatNode`, `ForNode`, dan `CaseNode`.

Struktur anotasi pada node AST menyimpan `typeCode`, `typeName`, `tabIndex`, `blockIndex`, `level`, dan status inisialisasi. Informasi ini diisi oleh semantic analyzer setelah AST selesai dibangun.

3.4 Syntax-Directed Translation

Konversi parse tree ke AST dilakukan oleh `ASTBuilder`. Setiap fungsi `visit` pada builder menerima node parse tree tertentu dan mengembalikan node AST yang relevan. Detail sintaksis yang tidak diperlukan, seperti `beginsy`, `endsy`, `semicolon`, dan node grammar perantara, tidak dimasukkan ke AST.

Tabel berikut merangkum sebagian semantic action yang digunakan.

Tabel 3.1: Contoh Semantic Action ASTBuilder

Produksi	Semantic Action
<code><program></code>	Membentuk <code>ProgramNode</code> , mengisi nama program dari header, daftar deklarasi dari declaration part, dan body dari compound statement.
<code><var-declaration></code>	Untuk setiap identifier pada identifier-list, membentuk satu <code>VarDeclNode</code> dengan type specification yang sama.
<code><assignment-statement></code>	Membentuk <code>AssignNode(target, value)</code> dari node variable dan expression.
<code><expression></code>	Jika terdapat relational operator, membentuk <code>BinOpNode</code> ; jika tidak, meneruskan hasil simple-expression.

Produksi	Semantic Action
<simple-expression>	Membentuk rantai BinOpNode untuk operator aditif dan UnaryOpNode untuk tanda minus di depan.
<factor>	Literal menjadi node literal, identifier menjadi VarRefNode, procedure/function call dalam expression menjadi FuncCallNode, dan not factor menjadi UnaryOpNode.
<array-type>	Membentuk ArrayTypeNode berisi index type dan element type.
<record-type>	Membentuk RecordTypeNode yang menyimpan field sebagai daftar VarDeclNode.

3.5 Implementasi Symbol Table

Symbol table diimplementasikan pada class `Semantic::SymbolTable`. Constructor symbol table membuat base scope dan mengisi predefined identifier. Scope dikelola dengan stack yang menyimpan `scopeLastStack` dan `scopeBlockStack`. Ketika masuk scope baru, program menambah entri `btab`; ketika keluar scope, stack dipop.

Registrasi identifier dilakukan oleh `registerIdentifier`. Fungsi ini mengecek redeklarasasi pada scope aktif, membuat entri `tab`, mengisi `link` ke identifier sebelumnya dalam scope yang sama, memperbarui `last` pada `btab`, dan menghitung ukuran parameter atau variabel lokal. Lookup dilakukan dari scope terdalam ke scope terluar.

Pseudocode Register Identifier

```
function registerIdentifier(name, obj, type, nrm, adr):  
    if name exists in current scope:  
        report redeclaration error  
    entry.identifier = name  
    entry.link = currentScopeLast  
    entry.obj = obj  
    entry.type = type  
    entry.lev = currentLevel  
    entry.adr = computed address  
    append entry to tab  
    current btab.last = new index
```

3.6 Implementasi Semantic Analyzer

Semantic analyzer melakukan traversal depth-first terhadap AST. Fungsi utama `analyze` menginisialisasi ulang symbol table, predefined procedure, struktur pendukung subrange, enum, record field, signature procedure/function, dan stack function aktif. Setelah itu `visitProgram` memproses deklarasi dan body program.

Setiap kategori node memiliki fungsi `visit`. Deklarasi diproses oleh `visitConstDecl`, `visitTypeDecl`, `visitVarDecl`, `visitProcedureDecl`, dan `visitFunctionDecl`. Statement diproses oleh `visitAssign`, `visitIf`, `visitWhile`, `visitRepeat`, `visitFor`, `visitCase`, dan `visitProcCall`. Ekspresi diproses oleh `visitExpression`, `visitVariable`, `visitConstant`, dan `visitFuncCall`.

3.6.1 Scope dan Block

Scope global dibuat saat symbol table diinisialisasi. Program didaftarkan sebagai object class `PROGRAM`. Main compound block dianalisis dalam scope baru sehingga output `btab` memiliki block terpisah untuk body utama. Procedure, function, dan record juga membuat scope baru untuk parameter, variabel lokal, atau field.

3.6.2 Type Resolution

Tipe dasar diselesaikan melalui lookup predefined type. Tipe array menambahkan entri ke `atab`. Tipe record membuat block baru untuk field. Tipe subrange menyimpan base type dan batas bawah/atas. Tipe enumerated menyimpan daftar identifier dan mendaftarkan setiap enumerator sebagai konstanta.

3.6.3 Type Checking

Type checking dilakukan pada ekspresi dan statement. Operator aritmatika memerlukan tipe numerik. Operator Boolean memerlukan Boolean. Operator relasional memerlukan operand kompatibel dan menghasilkan Boolean. Assignment memerlukan target yang assignable dan value yang assignment-compatible terhadap target.

3.6.4 Procedure dan Function

Procedure dan function disimpan dalam `procSignatures`. Saat pemanggilan terjadi, analyzer mengecek jumlah argumen, tipe argumen, dan kebutuhan by-reference parameter. Function call pada expression menghasilkan tipe return function. Untuk function declaration, analyzer juga memeriksa apakah body function menjamin assignment ke nama function sebagai nilai return.

3.7 Format Output

Output program Milestone 3 terdiri dari lima bagian: `tab`, `btab`, `atab`, parse tree, dan decorated AST. Label object class mencakup `program`, `const`, `variable`, `type`, `procedure`, dan `function`. Decorated AST menampilkan anotasi seperti `tab_index`, `type`, `lev`, dan `block`.

Bab 4

Pengujian

4.1 Strategi Pengujian

Pengujian dilakukan dengan membangun program menggunakan `make -B`, menjalankan `arion_parser` pada input Milestone 3, lalu membandingkan output aktual dengan file output yang tersimpan pada direktori `test/milestone-3`. Command yang digunakan adalah sebagai berikut.

Command Pengujian

```
make -B
for i in 1 2 3 4 5 6 7 8; do
    ./arion_parser test/milestone-3/input- $\$i$ .txt output- $\$i$ .txt
    diff -q output- $\$i$ .txt test/milestone-3/output- $\$i$ .txt
done
```

Seluruh test case yang digunakan berhasil menghasilkan exit code 0 dan output yang konsisten dengan expected output. Laporan ini menampilkan lima kasus utama yang mewakili deklarasi, ekspresi, control flow, structured type, dan subprogram.

Tabel 4.1: Ringkasan Pengujian Milestone 3

No	File Input	Fokus Pengujian
1	test/milestone-3/input-1.txt	Deklarasi variabel, assignment, ekspresi aritmatika, procedure call <code>writeln</code> , <code>tab/btab</code> .
2	test/milestone-3/input-2.txt	Operator aritmatika, <code>div</code> , <code>mod</code> , unary minus, <code>not</code> , dan kondisi Boolean.
3	test/milestone-3/input-3.txt	Control flow: <code>while</code> , <code>for downto</code> , dan <code>repeat-until</code> .
4	test/milestone-3/input-4.txt	Structured type: record, array, field access, array access, dan case statement.
5	test/milestone-3/input-5.txt	Procedure, function, parameter, local scope, recursive function call, dan function return.

4.2 Detail Pengujian

4.2.1 Kasus 1: Program Dasar

Input

```
program Hello;  
  
var  
  a, b: integer;  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.
```

Cuplikan Output

```
=== tab ===  
42      Hello      program      10      0      1      0      0      41  
43          a      variable      0      0      1      0      0      42  
44          b      variable      0      0      1      0      1      43  
  
=== btab ===  
0      44      0      0      2  
1      0      0      0      0  
  
=== Decorated AST ===  
Program('Hello') -> type:Unknown, lev:0, block:0  
  VarDecl('a') -> tab_index:43, type:Integer  
  VarDecl('b') -> tab_index:44, type:Integer  
  Block -> type:Unknown, lev:1, block:1  
    Assign  
    ProcCall('writeln')
```

Kasus ini menunjukkan bahwa program dasar dapat dianalisis secara semantik. Identifier program dicatat sebagai `program`, variabel `a` dan `b` masuk ke `tab`, block utama masuk ke `btabs`, dan decorated AST memiliki anotasi tipe serta block.


```
$ ./arion_parser test/milestone-3/input-1.txt test/milestone-3/output-1.txt

--- input-1.txt ---
program Hello;

var
  a, b: integer;
begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.

--- output-1.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer   type      0  0  1  0  0  0
34 Real      type      1  0  1  0  0  33
35 Char      type      2  0  1  0  0  34
36 Boolean   type      3  0  1  0  0  35
37 String    type      4  0  1  0  0  36
38 True      const     3  0  1  0  1  37
39 False     const     3  0  1  0  0  38
40 writeln   procedure 10  0  1  0  0  39
41 readln    procedure 10  0  1  0  0  40
42 Hello     program   10  0  1  0  0  41
43 a         variable 0  0  1  0  0  42
44 b         variable 0  0  1  0  1  43

=== btab ===
idx last lpar psze vsze
-----
0 44 0 0 2
1 0 0 0 0

=== atab ===
(empty)
...
```

Gambar 4.1: Bukti input dan output pengujian Kasus 1.

4.2.2 Kasus 2: Ekspresi dan Boolean

Input

```
program Expressions;

var
  x, y, z: integer;

begin
  x := (10 + 5) - 3 * 2;
  y := x div 2 + 1;
  z := -x mod 3;
  if not (x == 0) then
    writeln(y + z);
end.
```

Cuplikan Output

```
Program('Expressions') -> type:Unknown, lev:0, block:0
VarDecl('x') -> type:Integer
VarDecl('y') -> type:Integer
VarDecl('z') -> type:Integer
Block -> lev:1, block:1
  Assign x := (10 + 5) - 3 * 2
  Assign y := x div 2 + 1
  Assign z := -x mod 3
  If -> condition type:Boolean
    ProcCall('writeln')
```

Kasus ini menguji inferensi tipe untuk operator aritmatika, integer division, modulo, unary minus, dan operator not. Kondisi pada if berhasil dianotasi sebagai Boolean.

```
$ ./arion_parser test/milestone-3/input-2.txt test/milestone-3/output-2.txt

--- input-2.txt ---
program Expressions;

var
  x, y, z: integer;

begin
  x := (10 + 5) - 3 * 2;
  y := x div 2 + 1;
  z := -x mod 3;
  if not (x == 0) then
    writeln(y + z);
  end.

--- output-2.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer   type    0  0  1  0  0  0
34 Real      type    1  0  1  0  0  33
35 Char      type    2  0  1  0  0  34
36 Boolean   type    3  0  1  0  0  35
37 String    type    4  0  1  0  0  36
38 True      const   3  0  1  0  1  37
39 False     const   3  0  1  0  0  38
40 writeln   procedure 10  0  1  0  0  39
41 readln    procedure 10  0  1  0  0  40
42 Expressions program 10  0  1  0  0  41
43   x       variable  0  0  1  0  0  42
44   y       variable  0  0  1  0  1  43
45   z       variable  0  0  1  0  2  44

=== btab ===
idx last lpar psze vsze
-----
0 45 0 0 3
...
```

Gambar 4.2: Bukti input dan output pengujian Kasus 2.

4.2.3 Kasus 3: Control Flow

Input

```
program ControlFlow;

var
  i, sum: integer;

begin
  sum := 0;

  while sum < 100 do
  begin
    sum := sum + 1;
  end;

  for i := 10 downto 1 do
  begin
    sum := sum - i;
  end;

  repeat
    sum := sum + 5;
    i := i + 1;
  until sum >= 50;
end.
```

Cuplikan Output

```
Program('ControlFlow') -> type:Unknown, lev:0, block:0
VarDecl('i') -> type:Integer
VarDecl('sum') -> type:Integer
Block -> lev:1, block:1
  While -> condition type:Boolean
    Assign sum := sum + 1
  For('i', downto)
    Assign sum := sum - i
```

```
Repeat
  Assign sum := sum + 5
  Assign i := i + 1
  condition type:Boolean
```

Kasus ini memvalidasi bahwa semantic analyzer mampu menangani `while`, `for`, dan `repeat-until`. Ekspresi kondisi pada loop diuji agar menghasilkan tipe Boolean.

```
$ ./arion_parser test/milestone-3/input-3.txt test/milestone-3/output-3.txt

--- input-3.txt ---
program ControlFlow;

var
  i, sum: integer;

begin
  sum := 0;

  while sum < 100 do
  begin
    sum := sum + 1;
  end;

  for i := 10 downto 1 do
  begin
    sum := sum - i;
  end;

  repeat
    sum := sum + 5;
    i := i + 1;
  until sum >= 50;
end.
```

```
--- output-3.txt (snippet) ---
=== tab ===
idx id      obj      type  ref  nrm  lev  adr  link
... (reserved words 0-32)
33 Integer   type    0    0    1    0    0    0
34 Real      type    1    0    1    0    0    33
35 Char      type    2    0    1    0    0    34
36 Boolean   type    3    0    1    0    0    35
37 String    type    4    0    1    0    0    36
38 True      const   3    0    1    0    1    37
39 False     const   3    0    1    0    0    38
...
```

Gambar 4.3: Bukti input dan output pengujian Kasus 3.

4.2.4 Kasus 4: Structured Type

Input

```
program StructuredTypes;

type
  Point == record
    x, y: integer;
  end;

var
  p: Point;
  scores: array[1..5] of integer;
  grade: char;

begin
  p.x := 10;
  p.y := 20;
  scores[1] := p.x + p.y;

  grade := 'A';
  case grade of
    'A': writeln('Excellent');
    'B', 'C': writeln('Good');
  end;
end.
```

Cuplikan Output

```
=== atab ===
idx  xtyp  etyp  eref  low  high  elsz  size
   0     7     0     0     1     5     1     5

Program('StructuredTypes') -> type:Unknown
TypeDecl('Point') -> type:Record
  RecordType
    VarDecl('x')
    VarDecl('y')
VarDecl('p') -> type:Record
VarDecl('scores') -> type:Array
VarDecl('grade') -> type:Char
Block
  FieldAccess(.x)
  FieldAccess(.y)
  ArrayAccess scores[1]
  Case
```

Kasus ini menguji record, array, akses field, akses indeks array, serta case statement. Informasi array masuk ke atab, sedangkan field record disimpan melalui block record.

```
$ ./arion_parser test/milestone-3/input-4.txt test/milestone-3/output-4.txt

--- input-4.txt ---
program StructuredTypes;

type
  Point == record
    x, y: integer;
  end;

var
  p: Point;
  scores: array[1..5] of integer;
  grade: char;

begin
  p.x := 10;
  p.y := 20;
  scores[1] := p.x + p.y;

  grade := 'A';
  case grade of
    'A': writeln('Excellent');
    'B', 'C': writeln('Good');
  end;
end.

--- output-4.txt (snippet) ---
=== tab ===
idx id      obj      type  ref  nrm  lev  adr  link
-----
... (reserved words 0-32)
33 Integer  type    0    0    1    0    0    0
34 Real     type    1    0    1    0    0    33
35 Char     type    2    0    1    0    0    34
36 Boolean  type    3    0    1    0    0    35
37 String   type    4    0    1    0    0    36
38 True     const   3    0    1    0    1    37
39 False    const   3    0    1    0    0    38
...
```

Gambar 4.4: Bukti input dan output pengujian Kasus 4.

4.2.5 Kasus 5: Subprogram

Input

```
program Subprograms;

var
  result: integer;

procedure PrintTwice(msg: string; count: integer);
var i: integer;
begin
```

```
for i := 1 to count do
begin
  writeln(msg);
end;
end;

function Factorial(n: integer): integer;
begin
  if n <= 1 then
    Factorial := 1
  else
    Factorial := n * Factorial(n - 1);
  end;
end;

begin
  PrintTwice('hello', 3);
  result := Factorial(5);
  writeln(result);
end.
```

Cuplikan Output

```
Program('Subprograms') -> type:Unknown
VarDecl('result') -> type:Integer
ProcedureDecl('PrintTwice') -> block:1
  Param('msg': string)
  Param('count': integer)
  VarDecl('i') -> type:Integer
  For('i', to)
    ProcCall('writeln')
FunctionDecl('Factorial', returns: 'integer') -> type:Integer, block:2
  Param('n': integer)
  If
    Assign Factorial := 1
    Assign Factorial := n * Factorial(n - 1)
Block -> block:3
  ProcCall('PrintTwice')
  FuncCall('Factorial') -> type:Integer
  ProcCall('writeln')
```

Kasus ini menunjukkan bahwa procedure dan function dicatat dalam symbol table, parameter diproses pada scope subprogram, pemanggilan recursive function dikenali sebagai FuncCall, dan function Factorial memiliki assignment ke nama function sebagai return value.

```
$ ./arion_parser test/milestone-3/input-5.txt test/milestone-3/output-5.txt

--- input-5.txt ---
program Subprograms;

var
  result: integer;

procedure PrintTwice(msg: string; count: integer);
var i: integer;
begin
  for i := 1 to count do
  begin
    writeln(msg);
  end;
end;

function Factorial(n: integer): integer;
begin
  if n <= 1 then
    Factorial := 1
  else
    Factorial := n * Factorial(n - 1);
  end;
end;

begin
  PrintTwice('hello', 3);
--- output-5.txt (snippet) ---
=== tab ===
idx id      obj      type ref nrm lev adr link
-----
... (reserved words 0-32)
33 Integer  type    0  0  1  0  0  0
34 Real     type    1  0  1  0  0  33
35 Char     type    2  0  1  0  0  34
36 Boolean  type    3  0  1  0  0  35
37 String   type    4  0  1  0  0  36
38 True     const   3  0  1  0  1  37
...
```

Gambar 4.5: Bukti input dan output pengujian Kasus 5.

4.3 Pengujian Error Semantic

Selain kasus valid, program juga diuji secara manual terhadap input yang mengandung kesalahan semantik. Contoh kesalahan yang berhasil dideteksi adalah undeclared identifier, assignment dengan tipe tidak kompatibel, kondisi if non-Boolean, redeklarasasi identifier, index array dengan tipe salah, dan pemanggilan by-reference dengan literal.

```
Input:
program BadType;
var x: integer; s: string;
begin
  x := s;
end.

Output:
=== Semantic Errors ===
- Assignment types are incompatible
```

Bab 5

Penutup

5.1 Kesimpulan

Milestone 3 berhasil menambahkan tahap semantic analysis pada Arion Compiler. Program dapat menerima source code, token stream, atau parse tree, kemudian menghasilkan parse tree, AST, symbol table, dan decorated AST. Implementasi AST membuat representasi program menjadi lebih ringkas dibanding parse tree, sedangkan semantic analyzer menambahkan informasi tipe, scope, block, dan referensi symbol table.

Symbol table `tab`, `btabs`, dan `atab` telah diimplementasikan untuk menyimpan identifier, block, dan array. Program juga telah mendukung predefined identifier, registrasi deklarasi, lookup identifier, pengecekan redeklarasi, type checking, assignment compatibility, procedure/function call, array access, record field access, serta validasi kondisi control flow.

Berdasarkan pengujian, program berhasil menangani kasus dasar, ekspresi, control flow, structured type, dan subprogram. Program juga mampu melaporkan beberapa semantic error secara informatif tanpa crash.

5.2 Saran

Pengembangan berikutnya dapat menambahkan analisis data-flow yang lebih lengkap untuk mendeteksi penggunaan variabel sebelum inisialisasi pada seluruh jalur eksekusi. Selain itu, pengecekan string length, kompatibilitas record anonymous, dan validasi cakupan nilai subrange pada ekspresi non-literal dapat diperluas agar semakin dekat dengan bahasa Pascal-like yang lengkap.

Untuk tahap selanjutnya, decorated AST yang sudah tersedia dapat dimanfaatkan sebagai dasar intermediate representation atau code generation. Pengujian juga dapat diperluas dengan kasus error yang lebih banyak, termasuk nested procedure/function, array bertingkat, record di dalam array, dan kombinasi scope yang lebih kompleks.

Bab 6

Lampiran

6.1 Tautan GitHub

- Repository: <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026>
- Halaman release: <https://github.com/Vixrlie/KAI-Tubes-IF2224-2026/releases>

6.2 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Kontribusi
1	Jonathan Kris Wicaksono	13524023	AST: merancang node hierarchy AST, Syntax-Directed Translation untuk konversi parse tree ke AST, dan AST formatter untuk output Decorated AST.	25%
2	Vincent Rionarlie	13524031	Semantic Analyzer: implementasi visit functions untuk setiap node, type checking dan compatibility rules, inferensi tipe, anotasi node AST, dan control flow validation.	25%
3	Muhammad Aufar Rizqi Kusuma	13524061	Symbol Table: implementasi <code>tab</code> , <code>btab</code> , <code>atab</code> , manajemen scope push/pop, registrasi dan lookup identifier, serta inisialisasi predefined identifier.	25%
4	Bryan Pratama Putra Hendra	13524067	Laporan dan README: penulisan laporan lengkap dan README, penyusunan bagian teori, serta quality assurance.	25%

6.3 Cara Menjalankan Program

```
make -B
make run INPUT=test/milestone-3/input-1.txt
make run-output INPUT=test/milestone-3/input-1.txt OUTPUT=output-m3.txt
```

Bibliografi

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. 2nd edition. Addison-Wesley, 2007.
- [2] Andrew W. Appel. *Modern Compiler Implementation in C*. 2nd edition. Cambridge University Press, 1998.
- [3] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. 2nd edition. Morgan Kaufmann, 2011.
- [4] Kenneth C. Louden. *Compiler Construction: Principles and Practice*. PWS Publishing Company, 1997.
- [5] Terence Parr. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf, 2010.
- [6] Niklaus Wirth. *Compiler Construction*. Addison-Wesley, 1996.